
Projet de Fin d'Etude TI_DSI 08/25_26

Pour l'obtention de

Licence Appliquée : Technologies de l'Informatique
Spécialité: Développement des Systèmes d'Information

**Développement d'un système ERP pour
l'entreprise Rayhan**

RAYHANERP



Présenté et soutenu, le 01/06/2026 par :

GUENNARI ALI ET GHOULA MOHAMED

Encadrant Universitaire : **Mdme. Bourkhiss Dalel**

Année universitaire 2025-2026



DEDICACE

Je dédie cet accomplissement majeur, fruit de mois d'étude empirique, à la mémoire sacrée de ma mère. J'ai la profonde espérance que, depuis sa demeure éternelle, elle perçoit cet humble hommage comme le témoignage éternel de la reconnaissance de son fils, qui ne cesse de prier pour le salut de son âme. Puisse le Tout-Puissant nous réunira un jour dans Son vaste paradis.

Guennari Ali

Je dédie ce travail à mes chers parents, qui ont toujours été à mes côtés et m'ont soutenu tout au long de ces longues années d'études. En signe de reconnaissance, qu'ils trouvent ici l'expression de ma profonde gratitude pour tous leurs efforts et sacrifices pour me voir réussir.

Ghoula Mohamed



REMERCIEMENT

Ce mémoire de fin d'études constitue l'aboutissement d'un travail approfondi, clôture une période de recherche intense, combinant revue de littérature exigeante et une réalisation concrète sur terrain. La réalisation de cet ouvrage, rendue possible par la grâce d'Allah, a bénéficié de l'appui précieux de plusieurs collaborateurs que nous tenons à remercier chaleureusement. Nous exprimons tout d'abord notre profonde gratitude à notre encadreuse, Mme Dalel Bourkhiss, directrice de ce travail, pour sa disponibilité constante et la pertinence de ses conseils. Ses remarques constructives sur les ébauches de ce mémoire ont été déterminantes pour aboutir à la version finale. Nos remerciements s'adressent ensuite à Mr. Nabil Derouiche, dont l'assistance précieuse, la disponibilité et l'accompagnement tout au long de nos démarches ont été indispensables à la concrétisation de ce projet. Nous tenons à remercier également Mr. Ahmed Fekih, ainsi que l'ensemble du personnel de la société Rayhan, pour leur accueil chaleureux et leur collaboration durant notre étude de terrain. Enfin, que l'ensemble des enseignants et du personnel administratif de l'Institut Supérieur des Études Technologiques (ISET) de Tataouine trouvent ici l'expression de notre reconnaissance. Nous remercions également les membres du jury pour l'honneur qu'ils nous font en acceptant d'évaluer ce travail.



TABLE DE MATIERE



LISTE DE FIGURES



LISTE DE TABLES



INTRODUCTION GENERALE

Aujourd'hui, plusieurs entreprises de différentes spécialités sont prêtes à investir des sommes considérables dans l'implantation de nouvelles technologies logicielles. Ces entreprises ont pour finalité d'améliorer leurs services, d'accroître leur agilité et flexibilité, de réduire leurs coûts, d'augmenter leur production ainsi que de faire face aux défis du marché. Cependant, la diversification et la croissance des activités au sein des entreprises ont fait grandir le besoin d'intégrer plusieurs technologies. La tâche de gérer efficacement toutes ces activités s'avère ainsi de plus en plus complexe et difficile. Pour surmonter ses propres difficultés, une entreprise a besoin d'utiliser des outils optimisés qui s'adaptent à la réalisation de toutes les activités en offrant des fonctionnalités riches et utiles. Parmi ces outils, nous retrouvons les systèmes intégrés de gestion tels que les ERP (Entreprise Ressources Planning) Les ERP sont des outils de gestion et d'analyse permettant d'optimiser la diffusion de l'information en interne, d'améliorer les processus de gestion et d'automatiser les activités et les tâches de l'entreprise.

Il en va de même pour l'entreprise Rayhan de plasturgie qui souhaite optimiser sa gestion de production par un système d'information unique grâce à un progiciel de gestion intégré L'intérêt de notre projet est d'automatiser la gestion de la production à l'aide de l'ERP. Afin de développer ce projet, il est nécessaire de passer par une étape d'analyse des besoins, suivie d'une conception détaillée du projet, puis de se concentrer sur le développement de modules, en sachant répondre à tous exigences.

Pour la société Rayhana , le déploiement d'un ERP est un levier de digitalisation indispensable pour moderniser les processus et se conformer aux standards Nationaux et internationaux C'était une mission qui semblait importante pour la société, ce qui Nous a poussé à s'investir énormément pour la mener à bien et pouvoir en tirer des conclusions enrichissantes pour eux comme pour

Nous Le présent rapport est organisé en quatre chapitres :

Le premier chapitre est un cadrage Générale du projet qui présente un aperçu sur les différents concepts théoriques d'un ERP, Objectifs et Enjeux du Projet l'ERP au sein de la société Rayhan

Le second chapitre est une analyse et spécifications des besoins : Problématique, identifications des Acteurs et détail des besoins de l'application Dans le troisième chapitre, nous allons présenter une conception de l'application qui comprend l'architecture technique du système, choix de conception et Modelés utilisées pour structurer l'application Ce travail établit la feuille de route logicielle. En fixant les règles et la structure en amont, il facilite et fiabilise l'exécution technique du projet. Dans le quatrième Chapitre Finalement, nous dresserons le bilan de ce modeste travail dans la conclusion de ce rapport.

Présentation Générale

Contents

1.1	Introduction	3
1.2	Présentation de l'organisme d'accueil : Société Rayhan	3
1.2.1	Description de la société Rayhan	3
1.2.2	Organigramme Rayhan	4
1.3	Objectif du projet	4
1.3.1	Objectifs du système ERP	5
1.3.2	Objectifs Fonctionnels (Ce que le système fait)	5
1.3.3	Objectifs Non Fonctionnels (Comment le système le fait)	5
1.4	Les ERP	6
1.4.1	Définition	6
1.4.2	Architecture technique	6
1.4.3	Les principaux éditeurs Le marché de l'ERP	7
1.4.4	Cycle de vie	10
1.5	Conclusion	11

1.1 Introduction

Dans une gestion classique d'une entreprise, chaque service est doté de son propre système d'information et ses propres applications (comptabilité, gestion de paie, gestion des stocks, gestion commerciale, gestion de production...). Pour faire le lien entre ces différents systèmes, l'entreprise faisait développer des interfaces informatiques entre ses différents SI. Ce mode de fonctionnement coûtait très cher à l'entreprise et il est devenu inappropriée. Pour améliorer cette situation, les entreprises ont décidé d'implémenter des systèmes intégrés connus par les progiciels de gestion intégré « PGI » ou en anglais Entreprise Ressource Planning « ERP »

1.2 Présentation de l'organisme d'accueil : Société Rayhan

Rayhan est une société Tunisienne de plasturgie (Implanté à Tataouine) spécialisé dans la fabrication de Sac a Poubelle, Bretelles en plastique et autres solutions de Packaging. La société Assure des produits de haute qualité Adaptés aux besoins industriels et Domestique



Figure 1.1: Logo Entreprise Rayhan

1.2.1 Description de la société Rayhan

- Nom de la société : SUARL Rayhan Industrie du packaging plastique.
- Nom du propriétaire : Fekih Ahmed

- Structure : Mono-site (un seul site de production).
- Effectif : 7 employés (tous utilisateurs du système ERP).
- Type de production : Une seule chaîne de production.
- Adresse : cité ABBES Tataouine Nord Tataouine 3200 Tataouine.
- MF : 195135Q/A/C/0000

1.2.2 Organigramme Rayhan

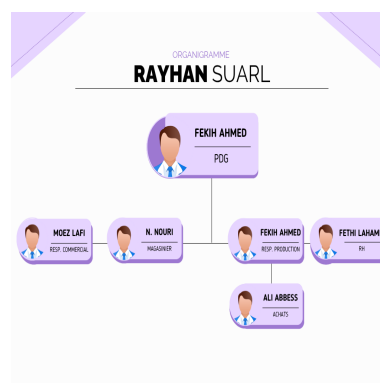


Figure 1.2: Organigramme

1.3 Objectif du projet

Mettre en place une solution ERP intégrée pour gérer l'ensemble des processus de L'entreprise : achats, ventes, stock, production, facturation, paie et gestion du Personnel.

Un ERP (Progiciel de Gestion Intégré) vise à centraliser les données dans une base unique pour automatiser les processus, améliorer la productivité et fiabiliser le pilotage stratégique en temps réel. Il standardise les opérations (ventes, RH, finance, production) pour accroître l'efficacité globale et la conformité.

1.3.1 Objectifs du système ERP

L'implantation d'un ERP (Progiciel de Gestion Intégré) dans une société industrielle comme Rayhan vise à unifier la gestion de la production (GPAO), des stocks et des ventes pour optimiser les performances. Ce processus nécessite une analyse des besoins, la configuration des modules, la reprise des données, la formation et un déploiement, souvent progressif, pour limiter les risques opérationnels

1.3.2 Objectifs Fonctionnels (Ce que le système fait)

- Centralisation des données : Regrouper toutes les informations dans une base unique, éliminant les silos.
- Automatisation des processus : Automatiser les tâches répétitives (facturation, gestion des stocks, saisie de commandes).
- Intégration métier : Lier les départements (ventes, production, comptabilité, ...) pour une collaboration fluide.
- Reporting et Business Intelligence : Générer des tableaux de bord et des indicateurs (KPI) pour la prise de décision.

1.3.3 Objectifs Non Fonctionnels (Comment le système le fait)

- Performance et Temps réel : Mettre à jour les informations instantanément.
- Sécurité : Renforcer la protection des données sensibles et la conformité réglementaire.
- Scalabilité (Évolutivité) : S'adapter à la croissance de l'entreprise et intégrer de nouvelles fonctionnalités.
- Ergonomie et Utilisabilité : Standardiser les interfaces pour faciliter l'adoption par les utilisateurs.

- **Disponibilité et Fiabilité** : Garantir un système robuste et accessible. Ces objectifs combinés permettent d'améliorer la rentabilité et de réduire les erreurs humaines. ,à améliorer l'efficacité, à fournir une visibilité sur les opérations et à permettre une prise de décision en temps réel. Les systèmes ERP offrent un large éventail de fonctionnalités, mais il est essentiel, pour réussir, de savoir quelles sont les fonctionnalités à rechercher et de comprendre comment mettre en œuvre un système ERP adapté aux besoins de Notre Entreprise « Rayhan »

1.4 Les ERP

1.4.1 Définition

Le concept d'ERP part d'un constat relativement simple selon lequel (la somme des optima est parfois inférieure 'a l'optimum de la somme).En d'autres termes, l'apport d'un ERP est toujours bien supérieur 'a la somme des apports de chacun des modules qui le composent. Cela pourrait se résumer par l'expression simple et bien connue La Définition Le terme ERP vient de l'anglais « Enterprise Resource Planning ». ERP a été traduit en français par l'acronyme PGI (Progiciel de Gestion Intégré) et se définit comme un groupe de modules relié à une base de données unique. L'ERP est un progiciel qui permet de gérer l'ensemble des processus opérationnels d'une entreprise en intégrant plusieurs fonctions de gestion dans un système : solution de gestion des commandes, solution de gestion des stocks, solution de gestion de la paie,solution de gestion de la comptabilité, solution de gestion d'e-commerce, solution de gestion de commerce B to B ou B to C Autrement dit, l'ERP représente la « colonne vertébrale » d'une entreprise.

1.4.2 Architecture technique

L'architecture d'un ERP se compose principalement d'un serveur ERP sur lequel est hébergée une base de données unique et disponible pour tous les utilisateurs. Elle implique l'utilisation de différents réseaux :

- Multi sites.
- Intranet, extranet.
- Serveur ERP à distance.

C'est la mise en réseau des données qui permettra leur gestion décentralisée, de plus, les ERP sont compatibles en packs Office. Enfin, les ERP sont aussi compatibles avec des outils de reporting

1.4.3 Les principaux éditeurs Le marché de l'ERP

Représente une vraie manne pour les prestataires de services informatiques. Il est considéré aujourd'hui, le marché le plus porteur de l'informatique. On distingue deux catégories d'ERP Les ERP Propriétaires et les ERP Open Source :

1.4.3.1 Les ERP Propriétaires

Aujourd'hui, de nombreux ERP propriétaires existent sur le marché. Nous évoquerons dans ce qui suit, quelques grands éditeurs :

- SAP (Business One) SAP/ est le leader mondial des ERP, est une application client-serveur. Ses modules couvrent l'ensemble des fonctions de gestion de l'entreprise et chaque module couvre des besoins complets de gestion. Ce progiciel a remporté rapidement un succès important auprès des grandes entreprises en proposant un progiciel multilingue et multidevise
- Oracle (JD Edwards) : Est un progiciel de gestion intégrée. Anciennement appelé People Enterprise One ou One World XE ou ERP 8, et vendu par J.D. Edwards puis par Peoplesoft. J.D. Edwards a été racheté par People Soft puis par Oracle. Le produit est depuis renommé "Oracle JDEdwards Enterprise One". Il est composé de plusieurs modules plus ou moins indépendants.

- ERP SAGE : SAGE Est un progiciel de gestion intégrée (ERP/PGI), conçu pour les structures de 20 à 500 employés, sociétés autonomes et filiales de groupes, des secteurs de l'industrie, du négoce et des services

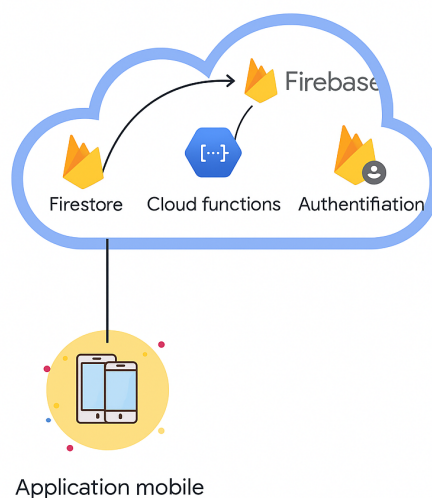
1.4.3.2 Les ERP Open Source

Ils sont relayés par des partenaires (SSII, Cabinets de Conseil) pour le support. L'implémentation d'un progiciel Open Source revient moins chère, puisqu'il n'y a pas de coût de licence. En revanche il faut inclure, dans le calcul du coût d'acquisition total, les frais de maintenance et de l'assistance technique. Voici la liste des principaux progiciels Open Source :

- Aria : Est le cœur de la gamme de produits Aria 4 XP, il couvre tous les domaines fonctionnels internes nécessaires pour gérer une entreprise. Il a été développé à base d'un ERP open source appelé Nola, et son environnement est PHPMYSQL
- Compiere : Est un progiciel de gestion intégré (PGI) et gestion de la relation client (GRC) à source ouverte, open source pour les Petites et moyennes entreprises (PME) dans la distribution et le service. L'application est fournie sous double licence GPL et propriétaire. Les sources peuvent être adaptées aux besoins du client. Le support technique et la documentation sont payants. Son origine est JorgJank
- ERP : Est un progiciel de gestion intégrée (ERP) libre, son origine est Nexedi. Son intégration avec NuxeoCPS, lui a permis d'être un système de gestion de contenu. L'environnement dans lequel il évolue est composé du Python et Zope
- Fistera : Est un PGI sous licence PGL, le premier client pour cet ERP était la société espagnole Auto Arte, son origine est Igalia, et son environnement est GNOME2 développement platform PostgreSQL
- Odoo : Anciennement OpenERP ou encore TinyERP, est un progiciel libre de gestion intégrée comprenant des modules de gestion des ventes, des relations clients, des projets, des entrepôts, de production, de comptabilité et de ressources humaines. Son environnement est constitué principalement du PostgreSQL et XML

Tableau Comparatif : ERP Propriétaire vs Open Source

Caractéristique ERP Open Source	Propriétaire ERP
Coût initial Faible à nul (pas de licence)	Élevé (licences, mise en place)
Coût à long terme Modéré (hébergement, accompagnement)	Élevé (entretien, permis)
Personnalisation Élevée,, code source modifiable	Limitée, souvent par l'éditeur
Dépendance Faible (indépendance)	Forte (fournisseur de verrouillage)

Table 1.1: Tableau descriptif du cas d'utilisation « S'authentifier »**Figure 1.3: Architecture 2-tiers**

Bien que ce schéma puisse paraître simpliste, une analyse détaillée des éléments de cette architecture et de leurs fonctions attachées met en évidence certains points essentiels sur lesquels on attire l'attention :

1. La fonction de présentation est à la charge du client exclusivement.
2. Le calcul (processing) est réparti entre le client et le serveur.

3. Le logiciel client est généralement spécifique au serveur.

4. Les données sont stockées ou accessibles via le serveur.

Dans le cadre d'une topologie d'accès à une base de données, le serveur traitera les requêtes en provenance du client qui se feront en général en langage SQL. C'est parce que le client assume des tâches de présentation et de processing, et donc de fait communique avec le serveur sans intervention d'un autre processus que le client est dit "lourd" par opposition à l'architecture 3-tiers où le client est constitué d'un simple navigateur internet et communique avec le serveur via un frontal.

1.4.4 Cycle de vie

Les applications mobiles ne sont pas des produits statiques qui peuvent être développés une fois puis oubliés. Ils nécessitent des mises à jour, des améliorations et une maintenance constantes pour suivre l'évolution des besoins et des attentes des utilisateurs et du marché. C'est là qu'intervient le concept de cycle de vie d'une application mobile. Le cycle de vie d'un logiciel désigne toutes les étapes du développement d'un logiciel, de sa conception à sa disparition. L'objectif d'un tel découpage est de permettre de définir des jalons intermédiaires permettant la validation du développement logiciel, c'est-à-dire la conformité du logiciel avec les besoins exprimés, et la vérification du processus de développement, c'est-à-dire l'adéquation des méthodes mises en œuvre. Pour le développement de notre application, nous avons choisis le modèle de cycle de vie en V comme l'indique la figure suivante :

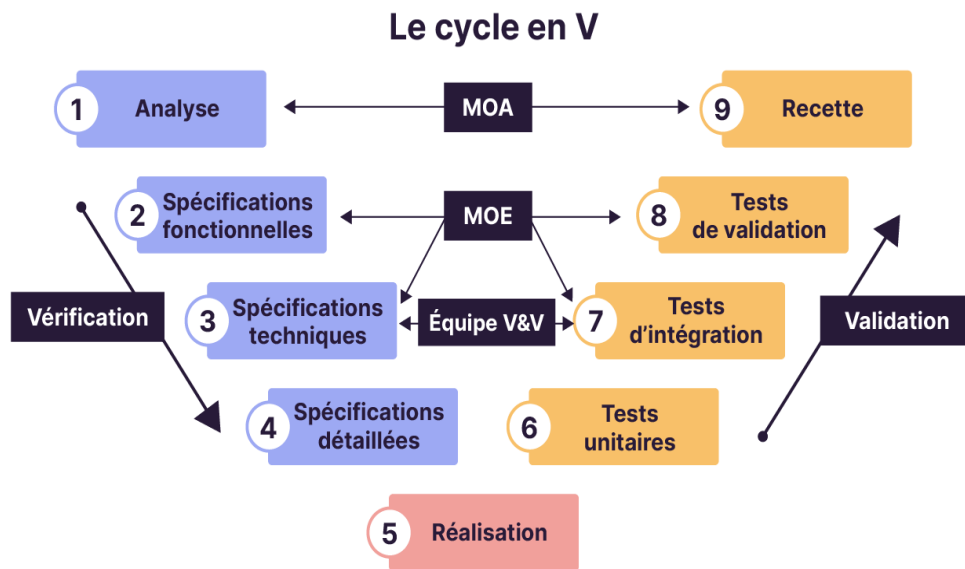


Figure 1.4: Modèle de cycle de vie en V

La méthode du cycle en V repose sur une idée simple. Les étapes de développement logiciel et de tests unitaires se répondent les unes aux autres. En clair, cela signifie que chaque phase de développement logiciel doit se conclure par une phase de tests unitaires. De plus, la mise en œuvre de cette méthode est simple. Une fois les spécifications fonctionnelles et les étapes de développement définies, chaque membre de l'équipe peut ainsi identifier ses tâches et voir où en est le produit. De même, le cycle en V ne nécessite pas de réunion quotidienne, seules quelques réunions de pilotage sont nécessaires lors du passage d'une phase à une autre.

1.5 Conclusion

Après avoir mettre le projet dans son cadre et définir ses objectifs, on a fait recours sur l'aspect organisationnel en spécifiant la méthodologie choisie pour le travail. Dans le prochain chapitre nous allons passer à la spécification et l'analyse des besoins.

Analyse et spécification des besoins

Contents

2.1	Introduction	13
2.2	Etude et critique de l'existant	13
2.3	Problématique	14
2.4	Solution proposée	14
2.5	Identification des besoins	15
2.5.1	Analyse des besoins fonctionnels	15
2.5.2	Analyse de besoins non fonctionnels	16
2.6	Diagramme de cas d'utilisation globale	16
2.6.1	Identification des acteurs	17
2.6.2	Identification des cas d'utilisation	17
2.6.3	Diagramme de cas d'utilisation global	18
2.7	Conclusion	19

2.1 Introduction

Un système ERP (Entreprise Resource Planning) est une solution informatique (logiciel/matériel) qui permet de gérer l'ensemble des processus d'une entreprise en intégrant l'ensemble de ses fonctions, dont la gestion des ressources humaines, la gestion comptable et financière, l'aide à la décision, la vente, la distribution, la relation client, la planification de la production et la gestion de la chaîne d'approvisionnement. C'est une suite des applications ou modules intégrés qu'une entreprise peut utiliser pour collecter, sauvegarder, gérer et interpréter l'ensemble des données venant des différentes activités de l'entreprise. Il s'agit de la gestion intégrée et centralisée des principaux processus et flux de travail (workflows) de l'entreprise, qui facilite la réalisation des objectifs commerciaux grâce à une vision courante et à jour des processus métier essentiels en fournissant une analyse en temps réel des opérations commerciales et des informations sur les indicateurs de performance globaux de l'entreprise, aidant ainsi l'organisation à planifier les étapes futures pour atteindre la croissance et le développement.

2.2 Etude et critique de l'existant

La plupart des solutions ERP existantes aujourd'hui cible les moyens à grands entreprises et les entreprises multinationaux, ce qui pose des difficultés pour les PME (Petites à Moyennes Entreprises). Principalement, on cite:

- Coût élevé et ressources limitées: les coûts des licences, mise en œuvre et maintenance sont souvent importants, même en mode SaaS, la migration, la formation et le temps alloué peuvent dépasser le budget.
- Complexité et durée de mise en place: la plupart des PME n'ont pas d'équipe IT dédiée, ce qui rend la mise en place du système ERP longue, compliqué et demandant en temps et ressources.

- Adaptation difficile aux processus métiers: Certains ERP imposent leur logique de fonctionnement, parfois menant à l'inadéquation avec les processus internes de l'entreprise surtout lorsque celles-ci sont très spécifiques ou simples.
- Adoption utilisateurs difficile et nécessité de formation: Les équipes peuvent mal utiliser l'ERP sans accompagnement suffisant conduisant à l'abandon de système et le retour aux anciens outils (Excel, mails...).

2.3 Problématique

TODO

2.4 Solution proposée

Nous voulons développer une solution ERP (application cross-platform) en faveur de l'entreprise Rayhanrépondant à ses besoins fonctionnels tout en veillant à la simplicité de l'usage et le mise en œuvre, des interfaces utilisateurs intuitives, facile à maintenir, compatible avec les workflows spécifiques et processus métier internes de l'entreprise, cout raisonnable et la possibilité d'expansion selon l'évolution du marché et les objectifs de l'entreprise.



Figure 2.1: Logo de l'application "RayhanERP"

2.5 Identification des besoins

TODO

2.5.1 Analyse des besoins fonctionnels

Les besoins fonctionnels représentent les actions essentielles que le système doit exécuter pour être opérationnel et répondre aux attentes des utilisateurs. Le système à concevoir devra satisfaire les besoins suivants :

2.5.1.1 TODO

- TODO
- Le Visiteur doit pouvoir rechercher des véhicules selon différents critères (marque, modèle, prix, kilométrage, etc.).
- Le Visiteur peut consulter les annonces détaillées avec photos et description.

2.5.1.2 Besoins liés au client

- Le client doit pouvoir sauvegarder des annonces dans une liste de favoris.
- Une messagerie intégrée doit permettre au client de discuter directement avec les vendeurs.

2.5.1.3 Besoins liés au TODO

- TODO
- TODO
- TODO
- TODO

2.5.1.4 Besoins liés à TODO

- TODO
- TODO

2.5.2 Analyse de besoins non fonctionnels

- **Performance TODO :**
 - TODO
 - TODO
- **Disponibilité TODO : TODO**
- **Fiabilité TODO**
 - TODO
 - TODO
- **Sécurité TODO**
 - TODO
 - TODO
- **Évolutivité TODO : TODO**
- **Ergonomie TODO : TODO**
- **Internationalisation TODO : TODO**

2.6 Diagramme de cas d'utilisation globale

L'étude de cas d'utilisation a pour objectif de déterminer les besoins des utilisateurs et les objectifs de l'application. La détermination des besoins est basée sur la représentation de

l'interaction fonctionnelle entre l'acteur et le système. Ainsi, il permet de donner une vision globale sur les interfaces de future application.

2.6.1 Identification des acteurs

- **Acteur** : un acteur est un utilisateur qui communique et interagit avec les cas d'utilisations du système. C'est une entité ayant un comportement comme une personne, système ou une entreprise. Nous citons ci-dessous les acteurs qui interagissent avec notre application :

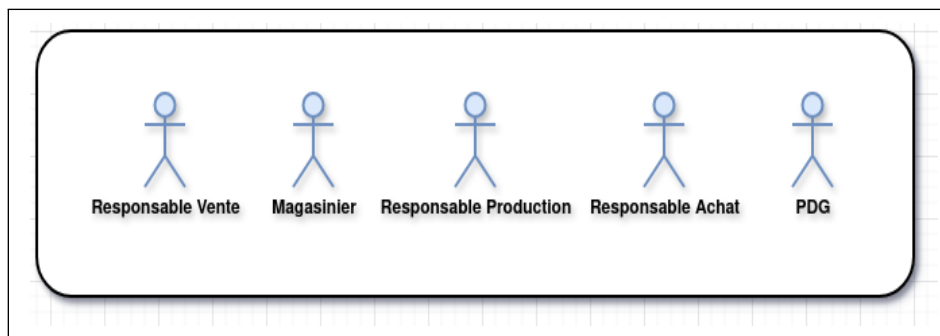


Figure 2.2: Les Acteurs de l'application

- **TODO** : TODO
- **TODO** : TODO
- **TODO** : TODO
- **TODO** : TODO

2.6.2 Identification des cas d'utilisation

Un cas d'utilisation représente un ensemble de séquences d'actions à réaliser par le système et produisant un résultat observable intéressant pour un acteur particulier. La liste ci-dessous représente les différents cas d'utilisation :

- **Cas d'utilisation « TODO »** : TODO
- **Cas d'utilisation « TODO »** : TODO

- Cas d'utilisation « TODO » : TODO
- Cas d'utilisation « TODO » : TODO
- ...TODO

2.6.3 Diagramme de cas d'utilisation global

Tout système peut être décrit par un certain nombre de cas d'utilisation correspondant aux besoins exprimés par l'ensemble des utilisateurs. A chaque utilisateur, vu comme acteur, correspond un certain nombre de cas d'utilisation du système. L'ensemble de ces cas se représente sous forme du diagramme ci-dessous :

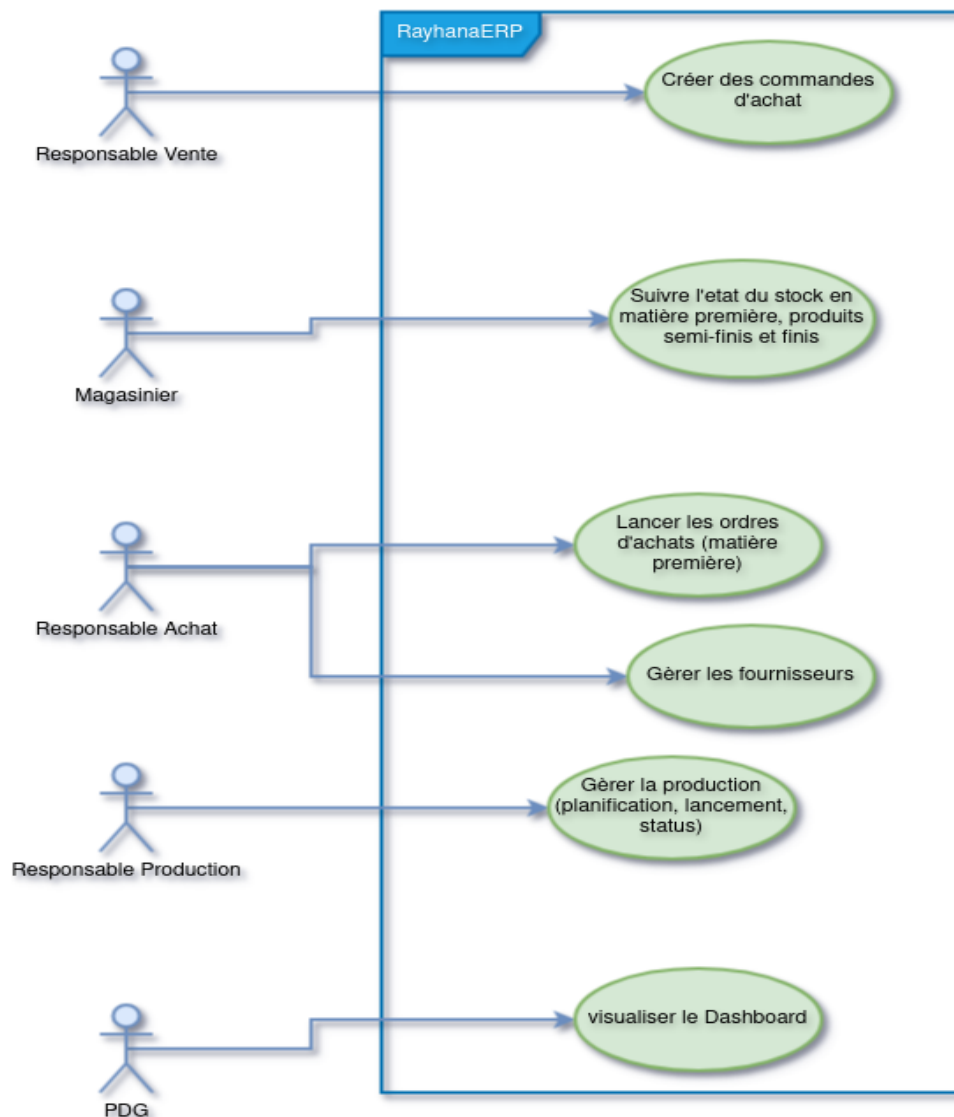


Figure 2.3: Diagramme de cas d'utilisation global de RayhanERP

2.7 Conclusion

Ce chapitre nous a permis de couvrir les différents besoins fonctionnels et non fonctionnels des acteurs de notre système et de construire le diagramme de cas d'utilisation global. Nous essayerons dans le chapitre qui suit de décortiquer ce Diagramme et concevoir clairement l'architecture de notre système.

CONCEPTION

Contents

3.1	Introduction	21
3.2	Choix de la méthodologie d'analyse	21
3.2.1	Présentation UML	21
3.2.2	Schémas de quelques diagrammes de cas d'utilisation	21
3.2.3	Diagramme de classe :	34
3.2.4	Diagramme de séquence :	35
3.2.5	Diagramme d'activité :	41
3.3	Conclusion	43

3.1 Introduction

Ce chapitre-ci est destiné à la conception qui est une étape primordiale dans le cycle de vie d'une application, elle a pour objectif d'élaborer à partir du modèle du système obtenu lors de l'étape d'analyse de besoin, des modèles détaillés de l'architecture du système. Le chapitre sera présenté comme suit : nous commençons par le choix de la méthode et les outils de modélisation. Ensuite nous détaillons les cas d'utilisation par diagrammes et description, puis nous présentons par ordre les diagrammes de séquence, les diagrammes de classe et finir par les diagrammes d'activité.

3.2 Choix de la méthodologie d'analyse

Pour élaborer cette application nous devons établir une conception pour atteindre le but final de notre projet. Pour cela nous avons choisi un langage de conception adapté avec nos besoins.

3.2.1 Présentation UML

UML (Unified Modeling Language), se définit comme un langage de modélisation graphique et textuel destiné à comprendre et à définir des besoins, spécifier et documenter des systèmes, esquisser des architectures logicielles, concevoir des solutions et communiquer des points de vue. UML modélise l'ensemble des données et des traitements en élaborant des différents diagrammes.

3.2.2 Schémas de quelques diagrammes de cas d'utilisation

Les cas d'utilisation représentent non seulement les interactions avec les acteurs, mais également les prés et post conditions ainsi que les enchaînements alternatifs. Dans cette section nous décrirons les cas d'utilisation des différents acteurs.

3.2.2.1 Cas d'utilisation : S'authentifier

Chaque utilisateur pour accéder à son compte et avoir ses fonctionnalités doit s'authentifier d'avance. La figure ci-dessous décrit ces relations :

RAYHANERP



Figure 3.1: Diagramme du cas d'utilisation «S'authentifier»

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation «S'authentifier» :

Titre	S'authentifier
Objectif	Vérifier l'identité de l'utilisateur afin de lui accorder l'accès à ses fonctionnalités selon son rôle.
Acteur	Visiteur, Client, Vendeur, Administrateur
Pré-condition	L'utilisateur possède un compte
Post-condition	L'utilisateur est authentifié et accède aux fonctionnalités qui lui sont dédiées
Scénario nominal	Saisir l'email et le mot de passe : -Le système vérifie les données saisies. -Le système envoi une requête de sélection à la base de données. -La base de données envoi les données au système -Le système vérifie la conformité de l'email et du mot de passe.
Scénario alternatif	Mot de passe ou email incorrecte : un message d'erreur apparait sous le champs correspondant.

Table 3.1: Tableau descriptif du cas d'utilisation « S'authentifier »

3.2.2.2 Cas d'utilisation : S'inscrire

La figure ce-dessous représente le cas d'utilisation « S'inscrire ».

RAYHANERP



Figure 3.2: Diagramme du cas d'utilisation «S'inscrire»

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation «S'inscrire» :

Titre	S'inscrire
Objectif	Permettre au visiteur de créer un compte.
Acteur	Visiteur
Pré-condition	L'utilisateur est un visiteur non connecté
Post-condition	Un nouveau compte est créé et le visiteur devient un client authentifié
Scénario nominal	-Le visiteur accède au formulaire d'inscription -Il saisit ses informations (nom, email, mot de passe, etc.) -Il valide le formulaire -Le système crée un nouveau compte utilisateur. -Un message de confirmation s'affiche
Scénario alternatif	Un email déjà utilisé, des données invalides : un message d'erreur apparaît sous le champs correspondant.

Table 3.2: Tableau descriptif du cas d'utilisation « S'inscrire »

3.2.2.3 Cas d'utilisation : Rechercher des véhicules

La figure ci-dessous représente le cas d'utilisation « Rechercher des véhicules ».



Figure 3.3: Diagramme du cas d'utilisation «Rechercher des véhicules

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation «Rechercher des véhicules» :

Titre	Rechercher des véhicules
Objectif	Permettre à l'utilisateur de rechercher des véhicules disponibles à la vente.
Acteur	Visiteur, Client
Pré-condition	L'utilisateur est sur la page d'accueil ou interface de recherche.
Post-condition	Les résultats de la recherche sont affichés
Scénario nominal	-L'utilisateur accède à l'interface de recherche de véhicules. -Le système affiche un formulaire de recherche avec différents critères. -L'utilisateur valide la recherche -Le système effectue une recherche dans la base de données selon les critères fournis. -Le système affiche les résultats -L'utilisateur applique des filtres supplémentaires.
Scénario alternatif	Aucun résultat ne correspond à la recherche : un message est affiché invitant à modifier les critères.

Table 3.3: Tableau descriptif du cas d'utilisation « Rechercher des véhicules »

3.2.2.4 Cas d'utilisation : Consulter des annonces

La figure ci-dessous représente le cas d'utilisation « Consulter des annonces ».



Figure 3.4: Diagramme du cas d'utilisation « Consulter des annonces »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation « Consulter des annonces » :

Titre	Consulter des annonces
Objectif	Permettre la consultation des annonces de véhicules publiées sur la plateforme.
Acteur	Visiteur, Client
Pré-condition	La plateforme contient des annonces.
Post-condition	L'utilisateur a visualisé une ou plusieurs annonces.
Scénario nominal	- L'utilisateur clique sur une annonce pour voir les détails. - Le système affiche l'annonce complète.
Scénario alternatif	Annonce indisponible : Message "Cette annonce n'est plus disponible"

Table 3.4: Tableau descriptif du cas d'utilisation « Consulter des annonces »

3.2.2.5 Cas d'utilisation : Ajouter aux favoris

La figure ci-dessous représente le cas d'utilisation « Ajouter aux favoris ».



Figure 3.5: Diagramme du cas d'utilisation « Ajouter aux favoris »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation « Ajouter aux favoris » :

Titre	Ajouter aux favoris
Objectif	Permettre au client d'ajouter une annonce à sa liste de favoris.
Acteur	Client
Pré-condition	Le client doit être authentifié.
Post-condition	L'annonce est enregistrée dans la liste des favoris du client.
Scénario nominal	- Le client sélectionne une annonce. - Il clique sur « Ajouter aux favoris ». - Le système vérifie l'authentification du client. - L'annonce est ajoutée aux favoris du client.
Scénario alternatif	- Si le client n'est pas authentifié, il est redirigé vers le formulaire de connexion. - Si une erreur survient, un message d'erreur est affiché.

Table 3.5: Tableau descriptif du cas d'utilisation « Ajouter aux favoris »

3.2.2.6 Cas d'utilisation : Contacter Vendeur

La figure ci-dessous représente le cas d'utilisation « Contacter Vendeur ».



Figure 3.6: Diagramme du cas d'utilisation « Contacter Vendeur »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation «Contacter Vendeur» :

Titre	Contacteur Vendeur
Objectif	Permettre au client d'envoyer un message ou une demande au vendeur.
Acteur	Client
Pré-condition	Le client doit être authentifié.
Post-condition	Le message est transmis au vendeur concerné.
Scénario nominal	- Le client consulte une annonce. - Il clique sur « Contacter le vendeur ». - Le système vérifie l'authentification du client. - Le client rédige son message. - Le système envoie le message au vendeur.
Scénario alternatif	- Si le client n'est pas authentifié, le système déclenche le cas d'utilisation « S'authentifier ». - En cas d'échec d'envoi, un message d'erreur est affiché.

Table 3.6: Tableau descriptif du cas d'utilisation « Contacter Vendeur »

3.2.2.7 Cas d'utilisation : Gérer Annonce

La figure ci-dessous représente le cas d'utilisation « Gérer Annonce ».



Figure 3.7: Diagramme du cas d'utilisation « Gérer Annonce »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation «Gérer Annonce» :

Titre	Gérer Annonce
Objectif	Permet au vendeur d'ajouter, modifier ou supprimer ses annonces publiées sur la plateforme.
Acteur	Vendeur
Pré-condition	- Le vendeur est authentifié. - Le vendeur a au moins une annonce existante pour modifier ou supprimer.
Post-condition	La base de données est mise à jour selon l'action effectuée (ajout/modification/suppression).
Scénario nominal	- Le vendeur choisit une action (ajouter/modifier/supprimer). - Le système valide et enregistre les modifications.
Scénario alternatif	- Ajout échoué : Champs obligatoires non remplis. - Modification échouée : Annonce verrouillée (en cours de modération). - Suppression échouée : Problème de connexion à la base de données.

Table 3.7: Tableau descriptif du cas d'utilisation « Gérer Annonce »

3.2.2.8 Cas d'utilisation : Gérer les utilisateurs

La figure ce-dessous représente le cas d'utilisation « Gérer les utilisateurs ».



Figure 3.8: Diagramme du cas d'utilisation « Gérer les utilisateurs »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation «Gérer les utilisateurs» :

Titre	Gérer les utilisateurs
Objectif	Permettre à l'administrateur de gérer les comptes utilisateurs (création, modification, suppression, consultation).
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié dans le système.
Post-condition	Les modifications apportées aux utilisateurs sont enregistrées dans le système.
Scénario nominal	- L'administrateur accède à l'interface de gestion des utilisateurs. - Le système affiche la liste des utilisateurs. - L'administrateur sélectionne une action (ajouter, modifier, supprimer, consulter). - Le système exécute l'action demandée et confirme son succès.
Scénario alternatif	- Si l'administrateur tente de supprimer un utilisateur inexistant, le système affiche un message d'erreur. - Si les données saisies pour un nouvel utilisateur sont invalides, le système demande de corriger les informations.

Table 3.8: Tableau descriptif du cas d'utilisation « Gérer les utilisateurs »

3.2.2.9 Cas d'utilisation : Valider les annonces

La figure ci-dessous représente le cas d'utilisation « Valider les annonces ».



Figure 3.9: Diagramme du cas d'utilisation « Valider les annonces »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation «Valider les annonces» :

Titre	Valider les annonces
Objectif	Permettre à l'administrateur de valider ou rejeter les annonces soumises par les utilisateurs avant leur publication.
Acteur	Administrateur
Pré-condition	- L'administrateur doit être authentifié. - Des annonces doivent être en attente de validation.
Post-condition	- Les annonces validées sont publiées. - Les annonces rejetées sont archivées ou supprimées.
Scénario nominal	- L'administrateur accède à l'interface de validation des annonces. - Le système affiche la liste des annonces en attente. - L'administrateur sélectionne une annonce et choisit de la valider ou de la rejeter. - Le système enregistre la décision et notifie l'utilisateur concerné si nécessaire.
Scénario alternatif	- Si l'administrateur rejette une annonce, il peut ajouter un commentaire expliquant la raison du rejet. - Si aucune annonce n'est en attente, le système affiche un message indiquant qu'aucune action n'est nécessaire.

Table 3.9: Tableau descriptif du cas d'utilisation « Valider les annonces »

3.2.2.10 Cas d'utilisation : Supprimer une annonce signalée

La figure ci-dessous représente le cas d'utilisation « Supprimer une annonce signalée ».

RAYHANERP



Figure 3.10: Diagramme du cas d'utilisation « Supprimer une annonce signalée »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation « Supprimer une annonce signalée » :

Titre	Supprimer une annonce signalée
Objectif	Permettre à l'administrateur de supprimer une annonce signalée comme inappropriée ou non conforme.
Acteur	Administrateur
Pré-condition	- L'administrateur doit être authentifié. - Une annonce doit avoir été signalée par un utilisateur ou le système.
Post-condition	- L'annonce signalée est supprimée du système.
Scénario nominal	- L'administrateur accède à la liste des annonces signalées. - Le système affiche les détails de l'annonce signalée. - L'administrateur confirme la suppression. - Le système supprime l'annonce et met à jour la base de données.
Scénario alternatif	- Si l'administrateur décide de ne pas supprimer l'annonce, il peut la marquer comme "non problématique" et la conserver. - Si l'annonce a déjà été supprimée ou n'existe pas, le système affiche un message d'erreur.

Table 3.10: Tableau descriptif du cas d'utilisation « Supprimer une annonce signalée »

3.2.2.11 Cas d'utilisation : Consulter Statistiques

La figure ci-dessous représente le cas d'utilisation « Consulter Statistiques ».

RAYHANERP



Figure 3.11: Diagramme du cas d'utilisation « Consulter Statistiques »

Tableau descriptif : Dans le tableau ci-dessous, nous allons décrire les détails du cas d'utilisation « Consulter Statistiques » :

Titre	Consulter les statistiques
Objectif	Permettre à l'administrateur d'accéder et d'analyser les données statistiques du système (ex : nombre d'utilisateurs, annonces publiées, signalements, etc.).
Acteur	Administrateur
Pré-condition	- L'administrateur doit être authentifié. - Les données statistiques doivent être disponibles et accessibles.
Post-condition	- L'administrateur visualise les statistiques demandées. - Les données restent inchangées (consultation seule).
Scénario nominal	- L'administrateur accède à l'interface des statistiques. - Le système affiche les options de filtrage (période, type de données, etc.). - L'administrateur sélectionne les critères et demande l'affichage. - Le système génère et présente les statistiques sous forme de graphiques ou de tableaux.
Scénario alternatif	Si aucune donnée n'est disponible pour les critères sélectionnés, le système affiche un message approprié.

Table 3.11: Tableau descriptif du cas d'utilisation « Consulter Statistiques »

3.2.3 Diagramme de classe :

3.2.3.1 Présentation :

Le diagramme de classe est considéré le plus important de la modélisation orienté objet, il montre la structure interne du système et il permet de fournir une représentation abstraite des objets du système qui vont interagir ensemble pour réaliser les cas d'utilisation. Il s'agit d'une vue statique car on ne tient pas compte du facteur temporel dans le comportement du système. Les principaux éléments de cette vue statique sont les classes et leurs relations : Une classe est une description d'un groupe d'objets partageant un ensemble commun de propriétés (les attributs), de comportement (les opérations ou méthodes) et de relations avec d'autres objets (les associations et les agrégations). Ci-dessous, le diagramme de classe de notre système :

RAYHANERP



Figure 3.12: Diagramme de classe de AutoSmart

3.2.4 Diagramme de séquence :

3.2.4.1 Présentation :

Les diagrammes de séquence, couramment utilisés par les développeurs, modélisent les interactions entre les objets dans un cas d'utilisation unique. Ils illustrent la manière dont les différentes parties d'un système interagissent entre elles pour exécuter une fonction, et l'ordre dans lequel les interactions se produisent lorsqu'un cas d'utilisation particulier est exécuté. En termes plus simples, un diagramme de séquence montre les différentes parties d'un système qui travaillent dans une "séquence" pour obtenir quelque chose. Les diagrammes de séquence sont une solution populaire de modélisation dynamique en langage UML, car ils se concentrent plus précisément sur les lignes de vie, les processus et les objets qui vivent simultanément, et les messages qu'ils échangent entre eux pour exercer une fonction avant la fin de la ligne de vie. Les diagrammes de séquence sont centrés sur le temps et ils montrent visuellement l'ordre de l'interaction en utilisant l'axe vertical du diagramme pour représenter le temps, quels messages sont envoyés et quand.

3.2.4.2 Diagrammes de séquence « Authentification »:

Pour s'authentifier, l'utilisateur accède vers l'interface d'authentification où il saisit son adresse e-mail et son mot de passe. Si l'un des champs est vide, l'utilisateur devra saisir à nouveau ces derniers. Le contrôleur vérifiera si les informations sont identiques, si la vérification aboutit l'utilisateur se voit autoriser l'accès et sera redirigé vers le page d'accueil, dans le cas contraire un message d'erreur sera affiché. La figure suivante le montre :

RAYHANERP



Figure 3.13: Diagramme de séquence « Authentification »

3.2.4.3 Diagrammes de séquence « Inscription »:

Au premier contact avec l'application l'utilisateur doit créer un compte en remplissant un formulaire d'inscription. Le diagramme de séquence relatif au cas d'utilisation "Inscription" est illustré par la figure suivante :

RAYHANERP



Figure 3.14: Diagramme de séquence « Inscription »

3.2.4.4 Diagrammes de séquence «Ajouter annonce»:

Le diagramme de séquence relatif au cas d'utilisation "Ajouter annonce" est illustré par la figure suivante :

RAYHANERP



Figure 3.15: Diagramme de séquence « Ajouter annonce »

3.2.4.5 Diagrammes de séquence «Modifier annonce»:

Le diagramme de séquence relatif au cas d'utilisation "Modifier annonce" est illustré par la figure suivante :

RAYHANERP



Figure 3.16: Diagramme de séquence « Modifier annonce »

3.2.4.6 Diagrammes de séquence «Contacter Vendeur»:

Le diagramme de séquence relatif au cas d'utilisation "Contacter Vendeur" est illustré par la figure suivante :

RAYHANERP



Figure 3.17: Diagramme de séquence « Contacter Vendeur »

3.2.4.7 Diagrammes de séquence «Ajouter aux favoris»:

Le diagramme de séquence relatif au cas d'utilisation "Ajouter aux favoris" est illustré par la figure suivante :

RAYHANERP



Figure 3.18: Diagramme de séquence « Ajouter aux favoris »

3.2.5 Diagramme d'activité :

3.2.5.1 Présentation :

Le diagramme d'activité (Activity Diagram) fait partie des diagrammes comportementaux. Il est utilisé pour modéliser les aspects dynamiques d'un système. Il s'agit de représenter les opérations d'un processus et leurs conséquences sur les objets (logiciels ou matériels). La modélisation peut être utilisée pour décrire le déroulement d'un cas d'utilisation ou d'une méthode. Les diagrammes d'activité affichent le flux de travail d'un point de départ à un point d'arrivée en détaillant les nombreux chemins de décision existant dans la progression des événements contenus dans l'activité. Ils peuvent être utilisés pour détailler des situations dans lesquelles un traitement parallèle peut avoir lieu lors de l'exécution de certaines activités. Les diagrammes d'activités sont utiles pour la modélisation métier car ils sont utilisés pour détailler les processus impliqués dans les activités métier.

3.2.5.2 Diagramme d'activité «Authentification» :

Le diagramme d'activité relatif au cas d'utilisation "Authentification" est illustré par la figure suivante :

RAYHANERP



Figure 3.19: Diagramme d'activité «Authentification»

3.2.5.3 Diagramme d'activité «Ajouter annonce» :

Le diagramme d'activité relatif au cas d'utilisation "Authentification" est illustré par la figure suivante :

RAYHANERP



Figure 3.20: Diagramme d'activité «Ajouter annonce»

3.3 Conclusion

Au cours de ce chapitre nous avons en premier lieu modélisé les diagrammes de cas d'utilisation en détaillant leurs enchainements, ensuite les diagrammes de séquences qui nous ont donné une vue approfondie sur les interactions entre les objets et cela nous a permis de réaliser le diagramme de classe. Vers la fin, nous présentons les diagrammes d'activité. Dans le prochain chapitre nous allons passer à la réalisation de notre application.

Réalisation

Contents

4.1	Introduction	45
4.2	Environnement de développement :	45
4.2.1	Environnement matériel :	45
4.2.2	Environnement logiciel :	45
4.2.3	Environnement technologie :	49
4.2.4	Gestion de base de données :	50
4.3	Implémentation du projet :	50
4.3.1	Interface d'accueil de l'application :	51
4.3.2	Interface Login :	52
4.3.3	Interface Inscription :	53
4.3.4	Interface Ajouter Annonce :	54

4.1 Introduction

Après avoir détaillé la conception adaptée à mon application dans le chapitre précédent, nous avons consacré le dernier chapitre de ce rapport à la partie réalisation. Pour cela nous allons présenter dans un premier lieu l'environnement matériel et logiciel de développement par la suite, nous décrirons la phase d'implémentation on se basant sur quelques imprimés écrans des interfaces de l'application.

4.2 Environnement de développement :

4.2.1 Environnement matériel :

Lors de la phase de développement nous avons disposé un seul ordinateur portable hp qui possède les caractéristiques suivantes :

- Marque : hp
- Modèle : Zbook G7
- Processeur : 10th Gen Intel(R) Core(TM) i7-10750H @ 2.60GHz
- Mémoire RAM : 8 Go
- Système d'exploitation : Windows 11 Pro.
- Type de système d'exploitation : 64 bits, processeur x64

4.2.2 Environnement logiciel :

Android Studio : est un environnement de développement pour développer des applications mobiles Android. Il est basé sur IntelliJ IDEA et utilise le moteur de production Gradle. Il peut être téléchargé sous les systèmes d'exploitation Windows, macOS, Chrome OS et Linux. En ce qui concerne les outils de création Flutter de premier ordre, Android Studio est à l'honneur. Il est

largement reconnu comme un environnement de développement intégré (IDE) incontournable pour la création d'applications Android.



Figure 4.1: Logo Android Studio

Lucidchart UML : est une plateforme de collaboration en ligne, basée sur le cloud, permettant la création de diagrammes et la visualisation de données, et autres schémas conceptuels.



Figure 4.2: Logo Lucidchart

Visual Studio Code : Visual Studio Code est un éditeur de code source léger et extensible développé par Microsoft pour Windows, Linux et macOS. Il s'exécute sur différents systèmes d'exploitation, cet éditeur de code supporte la majorité des langages de programmations existant, et il peut aussi exécuter du code grâce aux extensions créées par la communauté, ce qui lui permet de devenir un IDE complet. En plus de la capacité d'installer des extensions il embarque de base une puissante fonction permettant d'analyser et d'auto compiler le code appelée IntelliSense.



Figure 4.3: Logo de Visual Studio Code

OverLeaf : est une plateforme en ligne gratuite permettant d'éditer du texte en LATEX (Latex est un langage et un système de composition de documents permet de rédiger des documents dont la mise en page est réalisée automatiquement) sans aucun téléchargement d'application. En outre, elle offre la possibilité de rédiger des documents de manière collaborative



Figure 4.4: Logo de Overleaf

Git : c'est un système de contrôle de version gratuit et open source, créé à l'origine par Linus Torvalds en 2005. Contrairement aux anciens systèmes de contrôle de version centralisés tels que SVN et CVS, Git est distribué : chaque développeur dispose localement de l'historique complet de son dépôt de code. Cela rend le clonage initial du dépôt plus lent, mais les opérations ultérieures telles que "commit", "blame", "diff", "merge" et "log" sont beaucoup plus rapides. Git offre également un excellent support pour le branchement, la fusion et la réécriture de l'historique du dépôt, ce qui a conduit à de nombreux outils et flux de travail innovants et

puissants. Git est le système de contrôle de version le plus largement utilisé dans le monde aujourd'hui et est considéré comme la norme moderne pour le développement de logiciels.



Figure 4.5: Logo de Git

Canva : Canva est un outil de design graphique en ligne gratuit. Utilisez-le pour créer vos publications sur les réseaux sociaux, vos présentations, vos affiches, ..

RAYHANERP



Figure 4.6: Logo de Canva

GitHub : est une plateforme de développement qui permet aux développeurs de créer, stocker, gérer et partager leur code. Il utilise le logiciel Git, il est couramment utilisé pour héberger des projets de développement de logiciels open source. Le site assure également un contrôle d'accès et des fonctionnalités destinées à la collaboration comme le suivi des bugs, les demandes de fonctionnalités, la gestion de tâches et un wiki pour chaque projet. Le site est devenu le plus important dépôt de code au monde, utilisé comme dépôt public de projets libres ou dépôt privé d'entreprises.



Figure 4.7: Logo GitHub

4.2.3 Environnement technologie :

Dart : est un langage de programmation optimisé pour les applications sur plusieurs plateformes. Il est développé par Google et est utilisé pour créer des applications mobiles, de bureau, de serveur et web. Dart est un langage orienté objet, il est productif, rapide et portable, conçu pour être familier et donc accessible à de nombreux développeurs existants, grâce à ses aspects orientés objet et sa syntaxe qui (selon ses utilisateurs) permet à tout développeur C++, C, Objective-C ou Java d'être productif en quelques jours. Dart peut aussi être utilisé pour la programmation côté serveur, ainsi que le développement d'applications mobiles (via l'API Flutter).



Figure 4.8: Logo de Dart

Flutter : est un kit de développement de logiciels d'interface utilisateur open source créé par Google . Il peut être utilisé pour développer des applications multiplateformes à partir d'une seule base de code pour le Web , Fuchsia , Android , iOS , Linux , macOS et Windows. Flutter est la boîte à outils de l'interface utilisateur de Google permettant de créer de superbes applications nativement compilées pour mobile , Web et de bureau à partir d'une base de code unique.



Figure 4.9: Logo Flutter

4.2.4 Gestion de base de données :

Firestore : est un ensemble de services d'hébergement pour n'importe quel type d'application (Android, iOS, ...). Il propose d'héberger en · NoSQL :Not only Structured Query Language et en temps réel des bases de données, du contenu, de L'authentification sociale (Google, Facebook, Twitter et GitHub), et des notifications, ou encore des services, tel que par exemple un serveur de communication temps réel. Firestore a été lancé en 2011 sous le nom d'Envolv, par Andrew Lee et par James Templin.



Figure 4.10: Logo Firestore

4.3 Implémentation du projet :

Les interfaces doivent respecter les heuristiques d'utilité pour permettre à l'utilisateur un accès facile à ces interfaces afin de garantir une bonne compréhension des fonctionnalités de l'application. Nous présentons ici les interfaces les plus significatives de l'application.

4.3.1 Interface d'accueil de l'application :

Les deux figures suivantes représentent la page de démarrage et l'interface d'accueil de l'application :

RAYHANERP



Figure 4.11: Page de démarrage

RAYHANERP



Figure 4.12: Interface d'accueil

RAYHANERP



Figure 4.13: Menu

4.3.2 Interface Login :

La figure ci-dessous présente l'interface d'authentification dans laquelle on doit entrer l'adresse e-mail et le mot de passe pour commencer à utiliser notre application. Des messages appropriés sont affichés pour informer l'utilisateur en cas d'erreur d'authentification ou de succès de connexion. S'il n'a pas de compte, l'utilisateur clique sur s'inscrire pour passer à l'étape de création de compte.

RAYHANERP



Figure 4.14: Interface login AutoSmart

RAYHANERP



Figure 4.15: Initialiser mot de passe

4.3.3 Interface Inscription :

La figure 4.15 présente l'interface d'inscription où l'utilisateur soit qu'il est client ou transporteur doit remplir un formulaire pour avoir un compte dans l'application lui permettant de gérer toute les fonctionnalités.

RAYHANERP



Figure 4.16: Interface d'inscription AutoSmart

4.3.4 Interface Ajouter Annonce :

La figure ci-dessous présente l'interface Ajouter Annonce :

RAYHANERP



Figure 4.17: Interface Ajouter Annonce



CONCLUSION GENERALE

Le travail présenté est réalisé dans le cadre de notre projet de fin d'études en développement informatique. Il consiste en la création et la réalisation d'une application mobile et web dédiée à l'achat et à la vente de voitures en ligne. Notre application, nommée AutoSmart, propose une interface intuitive pour les vendeurs comme pour les acheteurs, facilitant ainsi les transactions automobiles et améliorant l'expérience utilisateur.

Avant de commencer ce projet, nous n'avions pas d'expérience en développement d'applications mobiles ni en développement web. Au début, nous avons consacré du temps à la recherche et à l'apprentissage du langage Dart ainsi que du framework Flutter. Grâce à cet effort, nous avons pu surmonter les difficultés initiales et progresser efficacement dans la réalisation du projet. Ce travail de conception et de développement de l'application AutoSmart a été très bénéfique pour nous sur plusieurs plans. Il nous a permis de consolider nos connaissances en programmation et en développement.

D'un point de vue technique, ce projet nous a permis de nous familiariser avec l'environnement du développement informatique et surtout de maîtriser le framework Flutter. Comme tout projet, et afin de rendre AutoSmart plus complet et attractif, plusieurs évolutions sont envisagées, telles que l'intégration d'un système de paiement sécurisé, d'un module de géolocalisation pour visualiser les voitures à proximité, ainsi que la mise en place d'un système de messagerie entre acheteurs et vendeurs.

Ce projet nous a permis d'en apprendre davantage sur le domaine du développement informatique, de mieux comprendre les exigences du monde professionnel, et de développer nos compétences en tant que futurs développeurs. Nous espérons que le travail accompli avec AutoSmart sera à la hauteur de la confiance qui nous a été accordée.



NETOGRAPHIE

[1] <https://isetta.rnu.tn/fra/pages/157/Presentation>

[2] <https://www.lucidchart.com/pages/fr/landing/>

[3] <https://code.visualstudio.com/>

[4] <https://flutter.dev/>

[5] <https://www.automobile.tn/fr>

[6] <https://karahba.tn/>

[7] <https://developer.android.com/studio>

[8] <https://firebase.google.com/>

[9] <https://fr.overleaf.com/>

AutoSmart

Jelliti Mohamed Riadh

Résumé :

Le présent travail effectué au sein de l'Institut Supérieur des Études Technologiques de Tataouine (ISETTA), s'inscrit dans le cadre du projet de fin d'étude pour l'obtention de la Licence Appliquée en Technologie de l'Informatique, spécialité Développement des Systèmes Informatiques. L'objectif de ce projet est de développer une application mobile nommée « AutoSmart », destinée à la vente et à l'achat de voitures en ligne. L'application doit être sécurisée, simple et facile à utiliser pour tous les groupes d'âge. Pour la développer, nous avons utilisé plusieurs technologies telles que le framework « Flutter » et le langage « Dart » pour coder l'application, le langage « UML » pour la réalisation de l'étude conceptuelle, et « Firebase » comme solution backend .

Mots clés : Flutter, Dart, UML, Firebase, MVC, MVVM.

Abstract :

This work, carried out within the Higher Institute of Technological Studies in Tataouine (ISETTA), is part of the final project for obtaining the Applied Licence in Computer Technology specializing in Computer Systems Development. The objective of this project is to develop a mobile application called AutoSmart, aimed at facilitating the buying and selling of cars online. The application must be secure, simple, and easy to use for all age groups. To develop it, we used several technologies such as the Flutter framework and the Dart language for coding the application, the UML language for carrying out the conceptual study, and Firebase as the backend solution.

Key-words : Flutter, Dart, UML, Firebase, MVC, MVVM.